

NAT GATEWAY-PRIVATE SUBNET INTERNET ACCESS SETUP

Prepared By: Kritika Agarwal

Project: AWS NAT Gateway Setup

Objective

- Login AWS
- Project Purpose

Architecture Diagram

- Structure
- Explanation

Tools & Services Used

- VPC
- Subnet
- Internet Gateway
- NAT Gateway
- Elastic IP
- Route Table
- EC2 Instance
- Security Group

Implementation Steps

- Command Testing
- Output

Conclusion

- Summary
- Learning Outcome

1. OBJECTIVE-

- Login to AWS Console.
- Search VPC
- Open the VPC Dashboard

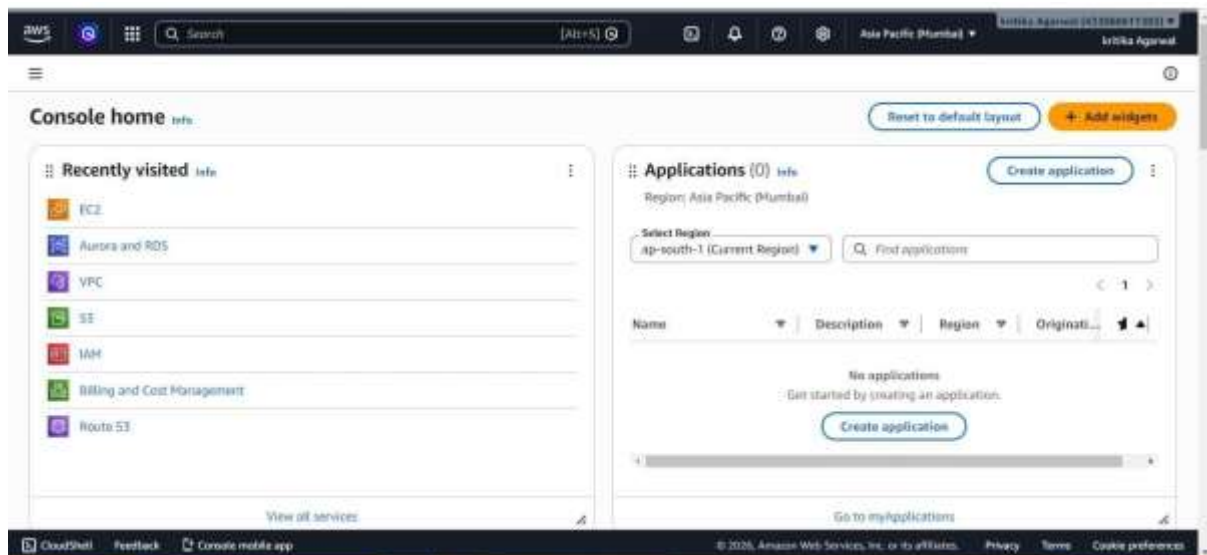


FIG: AWS CONSOLE HOME PAGE

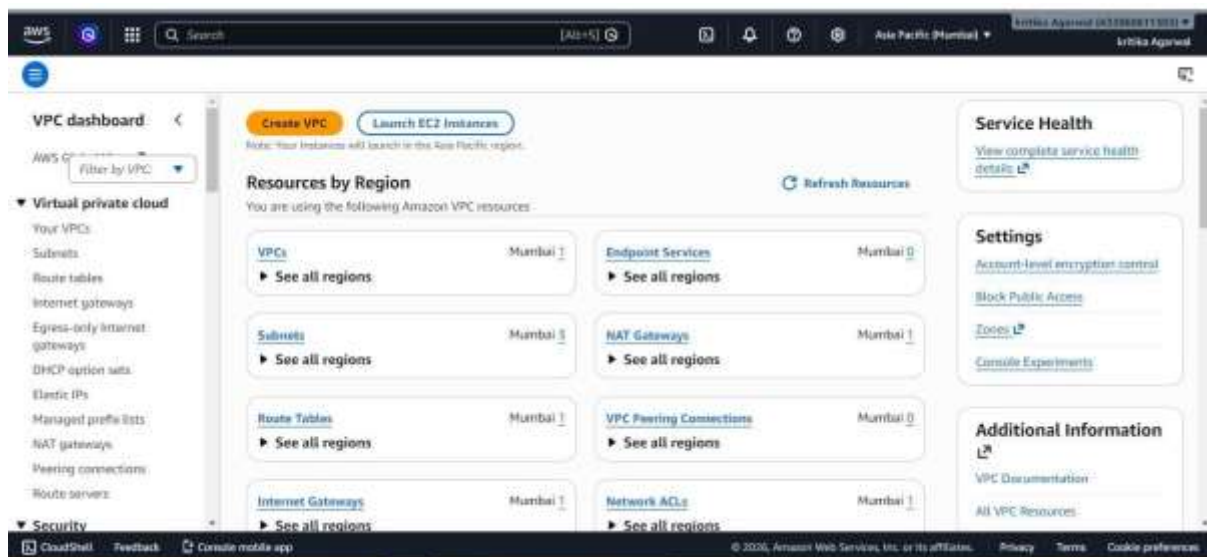


FIG:VPC DASHBOARD

2. ARCHITECTURE DIAGRAM-

- Internet: Provides internet connectivity.
- Internet Gateway: Connects the VPC to the internet.
- VPC: A private virtual network in AWS.
- Public Subnet: Hosts public resources like the Bastion Host and NAT Gateway.
- Bastion Host: Used to securely access the private EC2 instance.
- NAT Gateway: Provides outbound internet access to the private subnet.
- Elastic IP: A static public IP attached to the NAT Gateway.
- Private Route Table: Routes internet traffic to the NAT Gateway (0.0.0.0/0 → NAT Gateway).
- Private Subnet: Hosts secure EC2 instances without public IPs.

- EC2 Instance (Private): Can access the internet through the NAT Gateway but cannot be accessed directly from the internet.

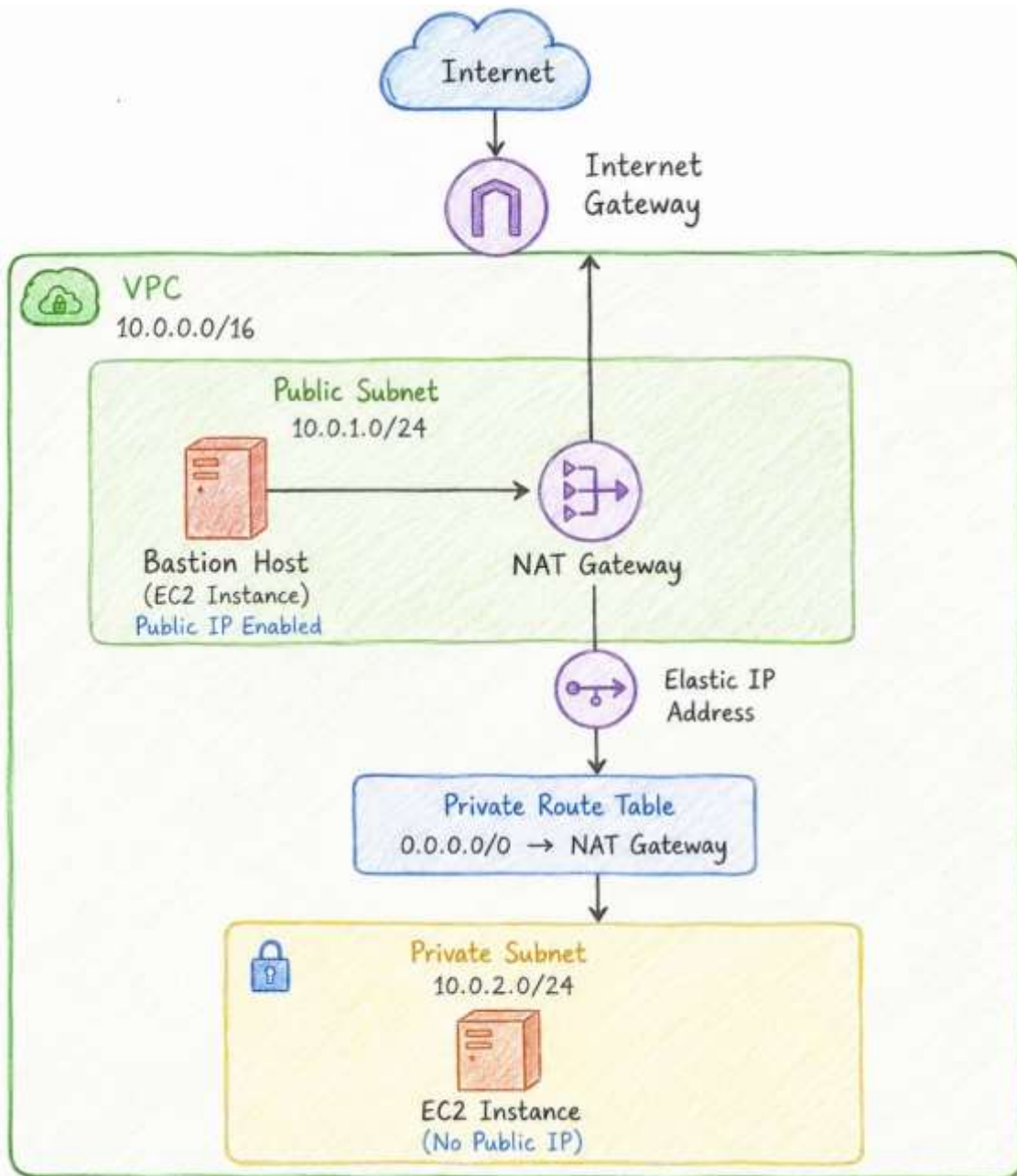


FIG: NAT GATEWAY

3. TOOLS & SERVICES USED-

STEP1: Create VPC

- Open VPC Dashboard.
- Click Create VPC.
- Enter-
 1. Name: Project-VPC
 2. CIDR:10.0.0.0/16
- Click Create VPC

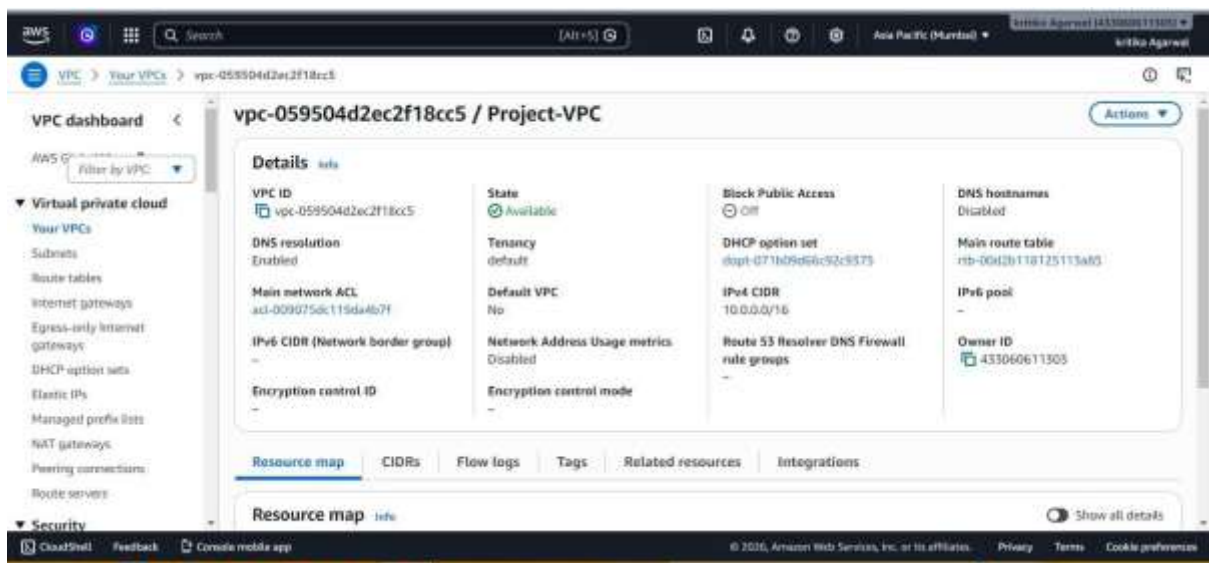


FIG:MY-VPC

STEP2:Create Public Subnet-

- Click Subnet
- Click Create Subnet
- Select Your VPC
- Name:Public Subnet
- CIDR: 10.0.1.0/24
- Select Availability Zone
- Click Create

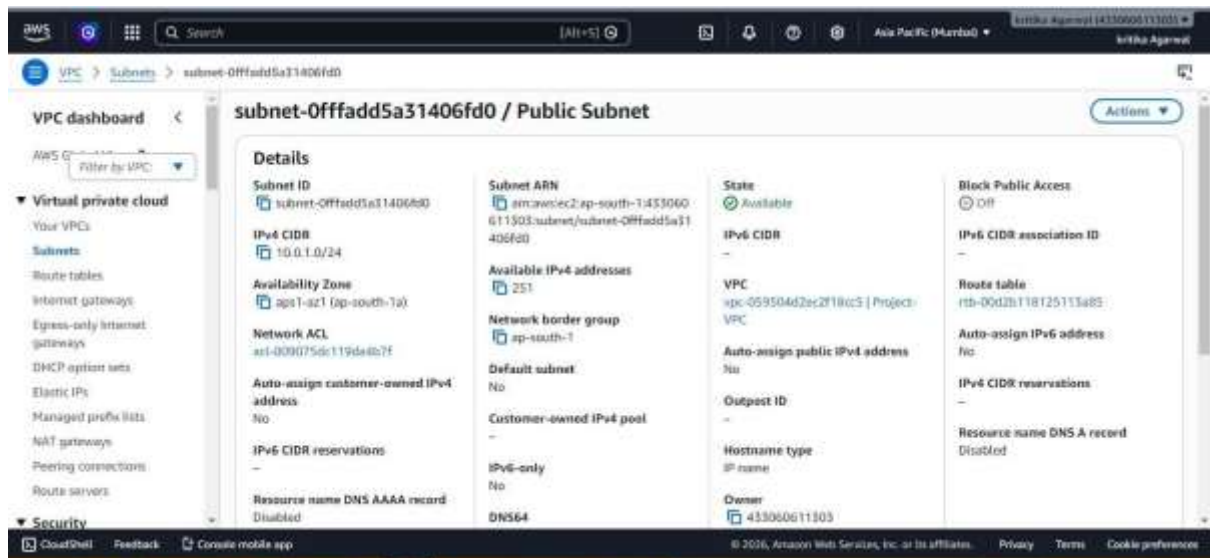


FIG: PUBLIC SUBNET

STEP3: Click Private Subnet-

- Click Subnet
- Click Create Subnet
- Select Your VPC
- Name: Private Subnet
- CIDR: 10.0.2.0/24
- Select Availability Zone
- Click Create

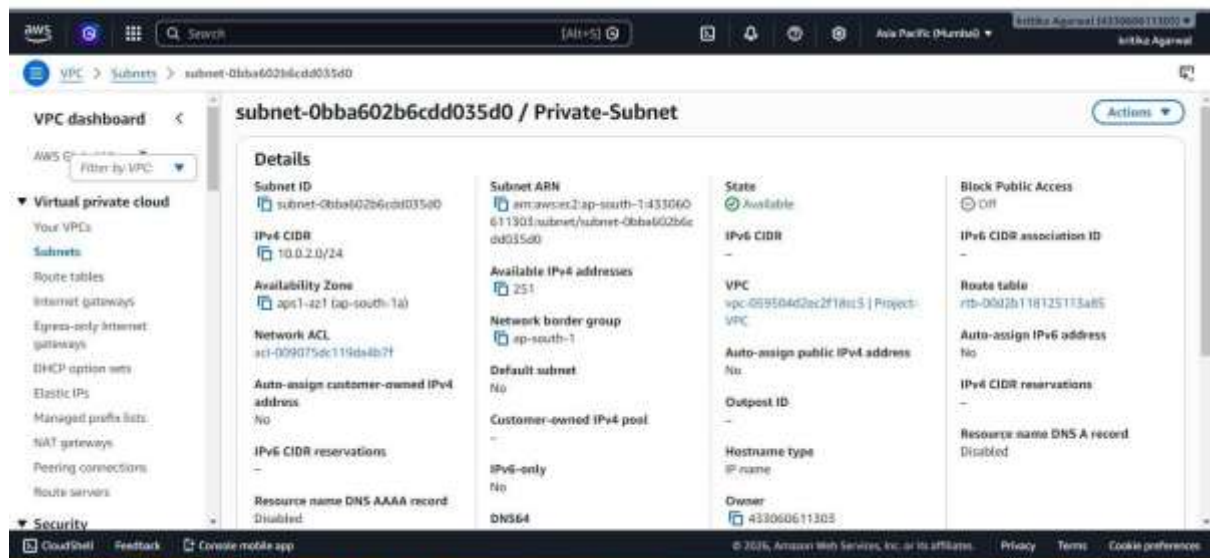


FIG: PRIVATE SUBNET

STEP4: Internet Gateway-

- Click Create internet gateway.
- Enter Name
- Click Create

- Click Attach to VPC
- Select your VPC.

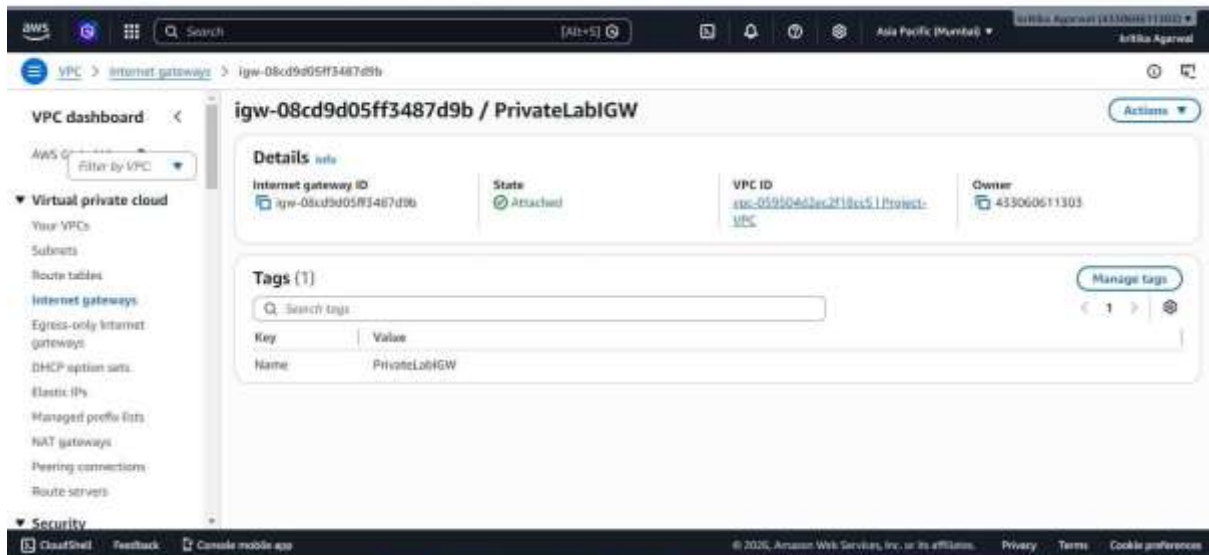


FIG: INTERNAT GATEWAY

STEP5:Nat Gateway:

- Open the VPC Dashboard.
- Click NAT Gateways.
- Click Create NAT Gateway.
- Enter-
 - (1)Name: my-nat-gateway
 - (2)Subnet: Public Subnet
 - (3)Connectivity Type: Public
 - (4)Elastic IP: Allocate a new Elastic IP
- ☑ Update Private Route Table-
 - (1)Destination:0.0.0.0/0
 - (2)Target: NAT Gateway

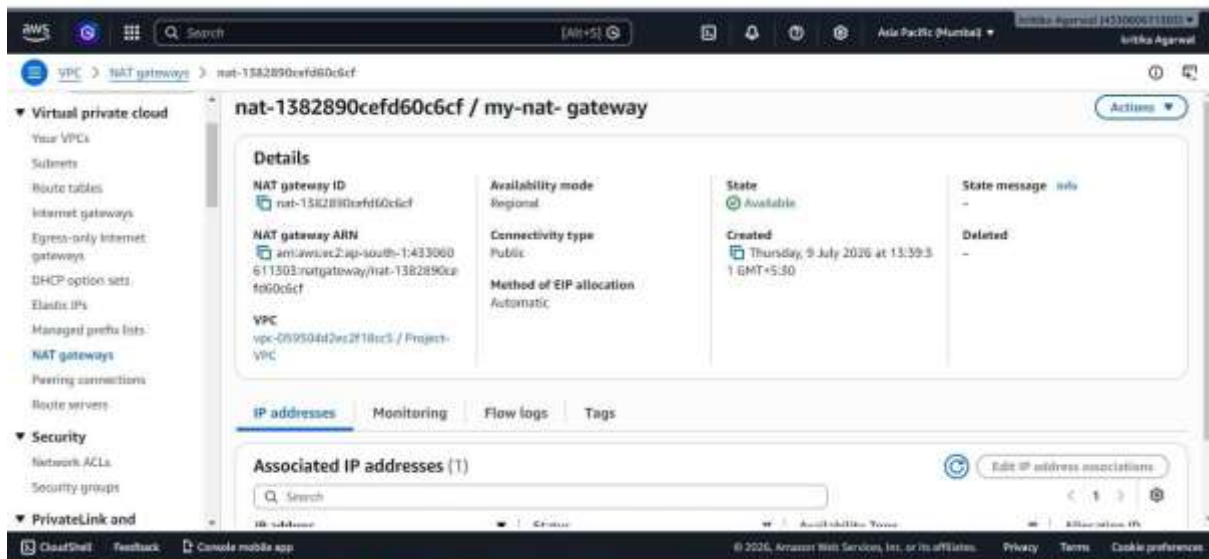


FIG: NAT GATEWAY

STEP6: Elastic IP Addresses:

- Automatically Created.
- Allocated easily.

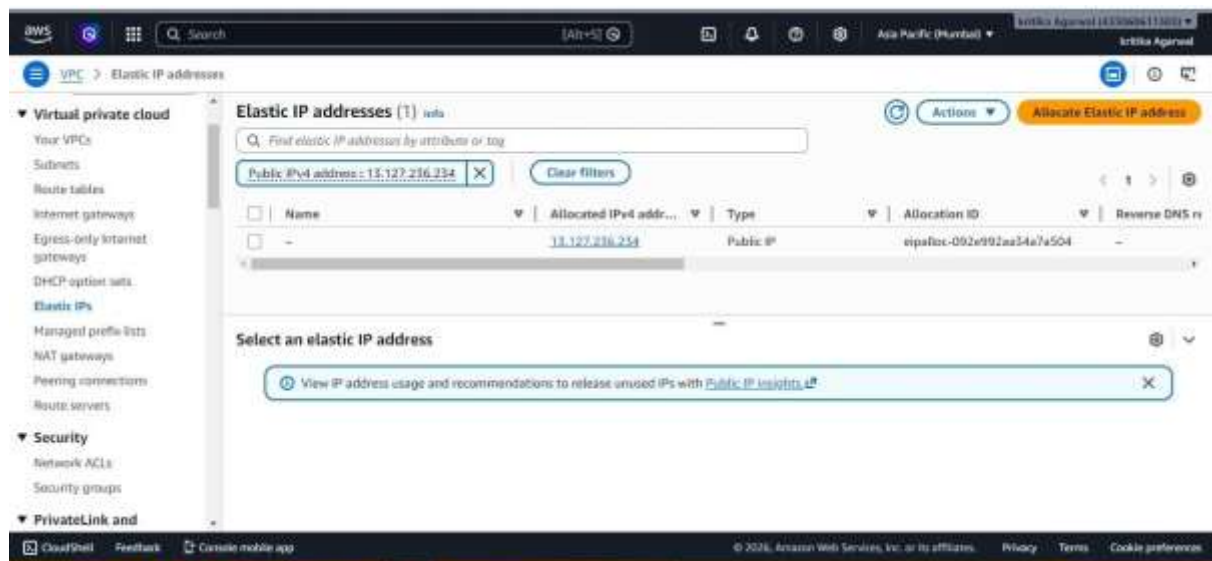


FIG: ELASTIC IP

STEP7: Route Table-

(i)- PUBLIC ROUTE TABLE:

- CONFIGURATION-
 - Associated with the Public Subnet.
 - Contains a route for the VPC CIDR (local route).

- Default route (0.0.0.0/0) points to the Internet Gateway (IGW).

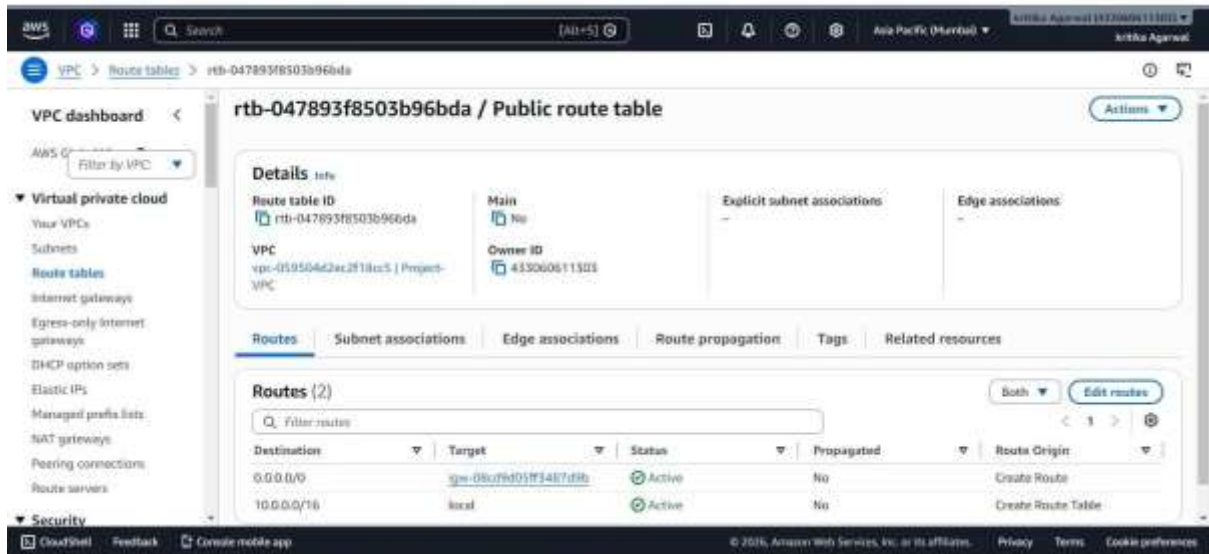


FIG: PUBLIC ROUTE TABLE

(ii)- PRIVATE ROUTE TABLE:

- CONFIGURATION-
 - Associated with the Private Subnet.
 - Includes a route to the peer VPC CIDR through the VPC Peering Connection.
 - No direct route to the Internet Gateway. If internet access is required, traffic is routed through a NAT Gateway.

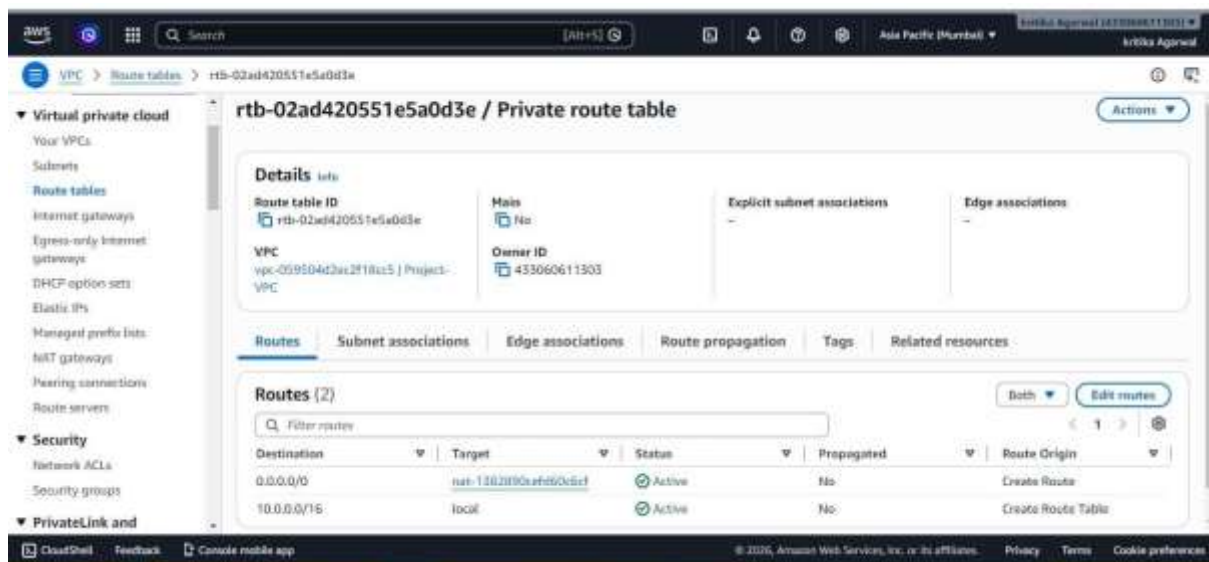


FIG: PRIVATE ROUTE TABLE

STEP8: EC2 Instance-

a. PUBLIC EC2 INSTANCE:

- INSTANCE NAME: Public-bastion
- AMI: Amazon Linux
- INSTANCE TYPE: t3.micro
- KEY PAIR: vpc.pem
- Bastion Host (Public EC2)

➤ NETWORK SETTING:

- VPC: Selected Project VPC
- SUBNET: Public Subnet
- AUTO-ASSIGN PUBLIC IP: Enabled
- SECURITY GROUP:
 - o SSH (Port 22)- Allowed from My IP
 - o All Traffic Allowed (for testing).

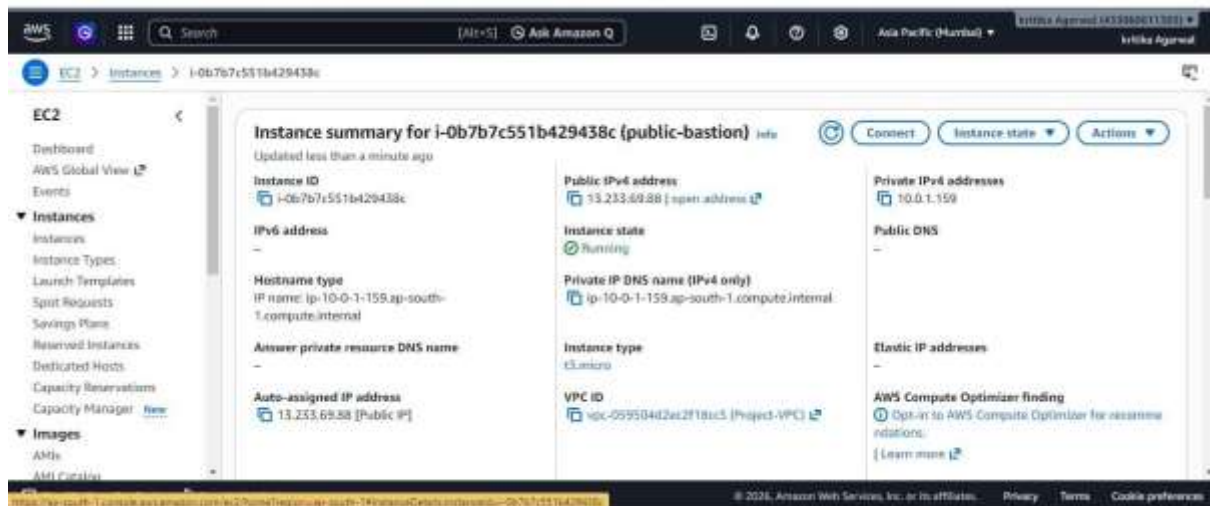


FIG: PUBLIC INSTANCE

b. PRIVATE EC2 INSTANCE:

- INSTANCE NAME: Private-ec2
- AMI: Amazon Linux
- INSTANCE TYPE: t3.micro
- KEY PAIR: Same Key Pair as Public EC2

➤ NETWORK SETTING:

- VPC: Selected Project VPC
- SUBNET: Private Subnet
- AUTO-ASSIGN PUBLIC IP: Disabled
- SECURITY GROUP:
 - o Allow SSH only from the Bastion Host Security Group (if using a bastion), or use AWS Systems Manager instead.

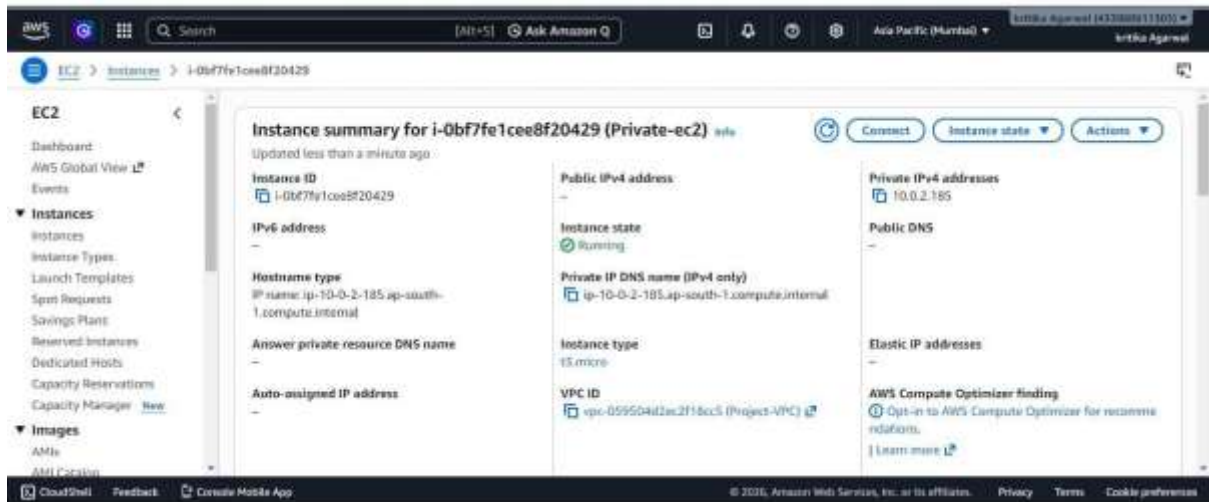


FIG:PRIVATE INSTANCE

4. IMPLEMENTATION STEPS-

➤ Command Testing-

1. STEPS:

- Connect to Private EC2:
 - Option 1 (Recommended): AWS Systems Manager (Session Manager)-
 - This avoids creating a bastion host. Ensure the EC2 has the required IAM role SSM .
 - Option 2: Bastion Host-
 - SSH into the Bastion Host, then SSH into the Private EC2 using its private IP.

2.STEPS:

- SSH INTO BASTION HOST
- Run command : cd Downloads
- `ssh -i "vpc.pem" ec2-user@<Public-IP>`


```

OpenSSH SSH client
Last metadata expiration check: 0:01:03 ago on Thu Jul 9 09:46:42 2026.
Dependencies resolved.

=====
Package                                Architecture Version                                Repository                               Size
=====
Installing:
httpd                                  x86_64      2.4.68-1.amzn2023.0.1                  amazonlinux                               46 k
Installing dependencies:
apr                                   x86_64      1.7.5-1.amzn2023.0.4                  amazonlinux                               129 k
apr-util                             x86_64      1.6.3-1.amzn2023.0.2                  amazonlinux                               97 k
apr-util-ldap                        x86_64      1.6.3-1.amzn2023.0.2                  amazonlinux                               13 k
generic-logos-httpd                 noarch      18.0.0-12.amzn2023.0.3                amazonlinux                               19 k
httpd-core                          x86_64      2.4.68-1.amzn2023.0.1                  amazonlinux                               1.4 M
httpd-filesystem                    x86_64      2.4.68-1.amzn2023.0.1                  amazonlinux                               12 k
httpd-tools                         x86_64      2.4.68-1.amzn2023.0.1                  amazonlinux                               80 k
libbrotli                           x86_64      1.0.9-4.amzn2023.0.2                  amazonlinux                               315 k
mailcap                             noarch      2.1.49-3.amzn2023.0.3                amazonlinux                               33 k
Installing weak dependencies:
apr-util-openssl                    x86_64      1.6.3-1.amzn2023.0.2                  amazonlinux                               15 k
mod_http2                           x86_64      2.0.42-1.amzn2023.0.1                  amazonlinux                               167 k
mod_lua                             x86_64      2.4.68-1.amzn2023.0.1                  amazonlinux                               59 k
=====
Transaction Summary
=====
Install 13 Packages

Total download size: 2.4 M
Installed size: 7.0 M
Downloading Packages:
(1/13): apr-1.7.5-1.amzn2023.0.4.x86_64. 0% [=====] --- B/s 0 B --- ETA
(1/13): apr-1.7.5-1.amzn2023.0.4.x86_64.rpm [=====] 3.2 MB/s 129 kB 00:00 ETA
(2-3/13): apr-util-ldap-1.6.3-1.amzn2023.0.2.x86_64.rpm [=====] 3.2 MB/s 13 kB 00:00 ETA
(2/13): apr-util-ldap-1.6.3-1.amzn2023.0.2.x86_64.rpm [=====] 3.2 MB/s 13 kB 00:00 ETA
(3-4/13): apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64.rpm [=====] 2.1 MB/s 97 kB 00:00 ETA
(3/13): apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64.rpm [=====] 3.2 MB/s 228 kB 00:00 ETA
(4-5/13): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch.rpm [=====] 590 kB/s 15 kB 00:00 ETA
(4/13): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch.rpm [=====] 3.2 MB/s 15 kB 00:00 ETA
(5-6/13): httpd-2.4.68-1.amzn2023.0.1.x86_64.rpm [=====] 3.2 MB/s 255 kB 00:00 ETA
(5/13): httpd-2.4.68-1.amzn2023.0.1.x86_64.rpm [=====] 1.8 MB/s 46 kB 00:00 ETA
(6-7/13): httpd-core-2.4.68-1.amzn2023.0.1.x86_64.rpm [=====] 3.2 MB/s 300 kB 00:00 ETA
(6/13): httpd-core-2.4.68-1.amzn2023.0.1.x86_64.rpm [=====] 598 kB/s 19 kB 00:00 ETA
(7-8/13): httpd-filesystem-2.4.68-1.amzn2023.0.1.x86_64.rpm [=====] 3.2 MB/s 319 kB 00:00 ETA
(7/13): httpd-filesystem-2.4.68-1.amzn2023.0.1.x86_64.rpm [=====] 3.0 MB/s 80 kB 00:00 ETA
(8-9/13): httpd-tools-2.4.68-1.amzn2023.0.1.x86_64.rpm [=====] 3.5 MB/s 1.8 MB 00:00 ETA
(8/13): httpd-tools-2.4.68-1.amzn2023.0.1.x86_64.rpm [=====] 31 MB/s 1.4 MB 00:00 ETA
(9-10/13): libbrotli-1.0.9-4.amzn2023.0.2.x86_64.rpm [=====] 3.5 MB/s 1.8 MB 00:00 ETA
(9/10/13): libbrotli-1.0.9-4.amzn2023.0.2.x86_64.rpm [=====]

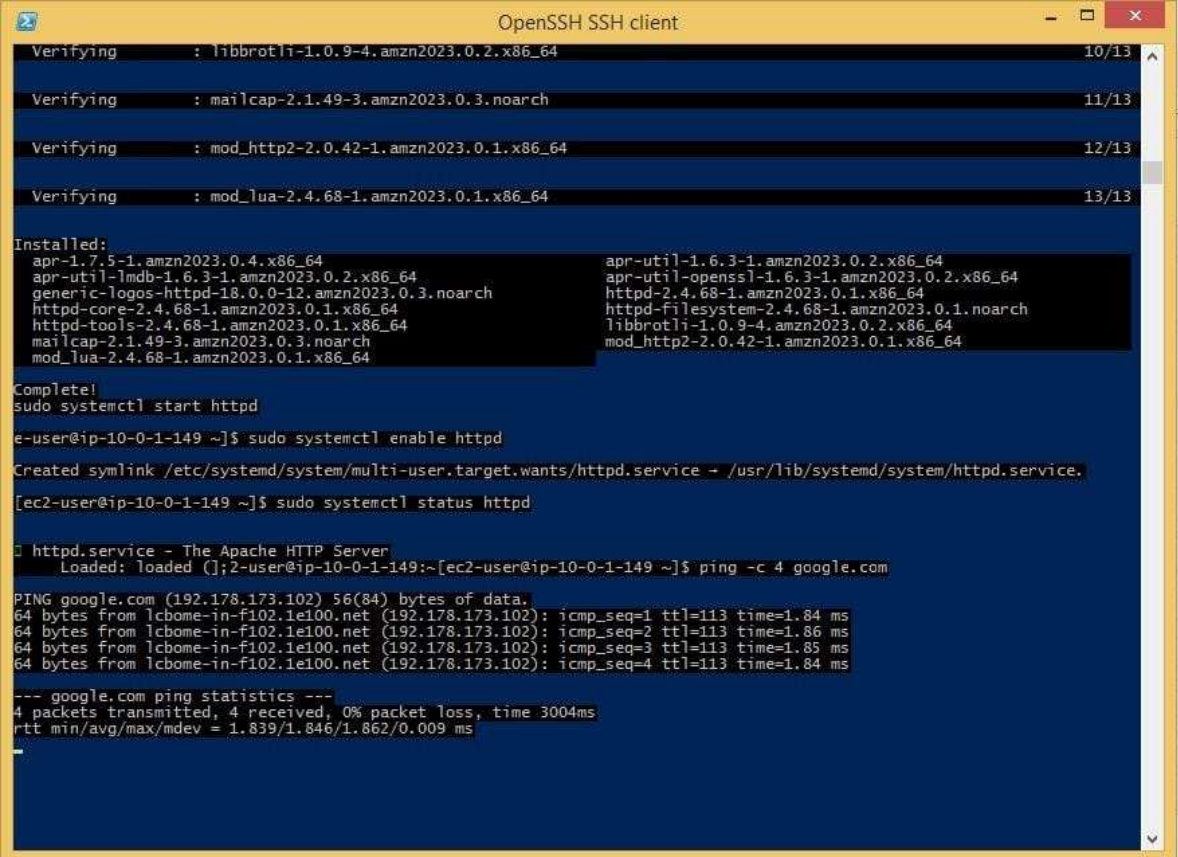
```

FIG: PACKAGE INSTALL

4.STEPS:

- SSH into Private EC2-
- FIND THE PRIVATE IP : 10.0.2.192
- From the bastion host: ssh -i vpc.pem ec2-user@10.0.2.192
- Test internet access-
- Inside the private ec2
- Ping google.com
- Update Packages-
- Amazon linux 2023
- Command used : dnf update -y

FIG: PRIVATE EC2



```
OpenSSH SSH client
Verifying      : libbrotli-1.0.9-4.amzn2023.0.2.x86_64      10/13
Verifying      : mailcap-2.1.49-3.amzn2023.0.3.noarch      11/13
Verifying      : mod_http2-2.0.42-1.amzn2023.0.1.x86_64    12/13
Verifying      : mod_lua-2.4.68-1.amzn2023.0.1.x86_64     13/13

Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64
apr-util-1.6.3-1.amzn2023.0.2.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
httpd-core-2.4.68-1.amzn2023.0.1.x86_64
httpd-tools-2.4.68-1.amzn2023.0.1.x86_64
mailcap-2.1.49-3.amzn2023.0.3.noarch
mod_lua-2.4.68-1.amzn2023.0.1.x86_64
apr-util-1.6.3-1.amzn2023.0.2.x86_64
apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
httpd-2.4.68-1.amzn2023.0.1.x86_64
httpd-filesystem-2.4.68-1.amzn2023.0.1.noarch
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
mod_http2-2.0.42-1.amzn2023.0.1.x86_64

Complete!
sudo systemctl start httpd

ec2-user@ip-10-0-1-149 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.

[ec2-user@ip-10-0-1-149 ~]$ sudo systemctl status httpd
httpd.service - The Apache HTTP Server
Loaded: loaded (/usr/lib/systemd/system/httpd.service; vendor preset: enabled)
Active: active (running) since Wed 2023-10-11 14:54:14 UTC; 1min 45s ago
Main PID: 1000
Tasks: 10 (limit: 1024)
Memory: 1.1M
CGroup: /systemd/system/httpd.service
└─ 1000 /usr/sbin/httpd -DFOREGROUND

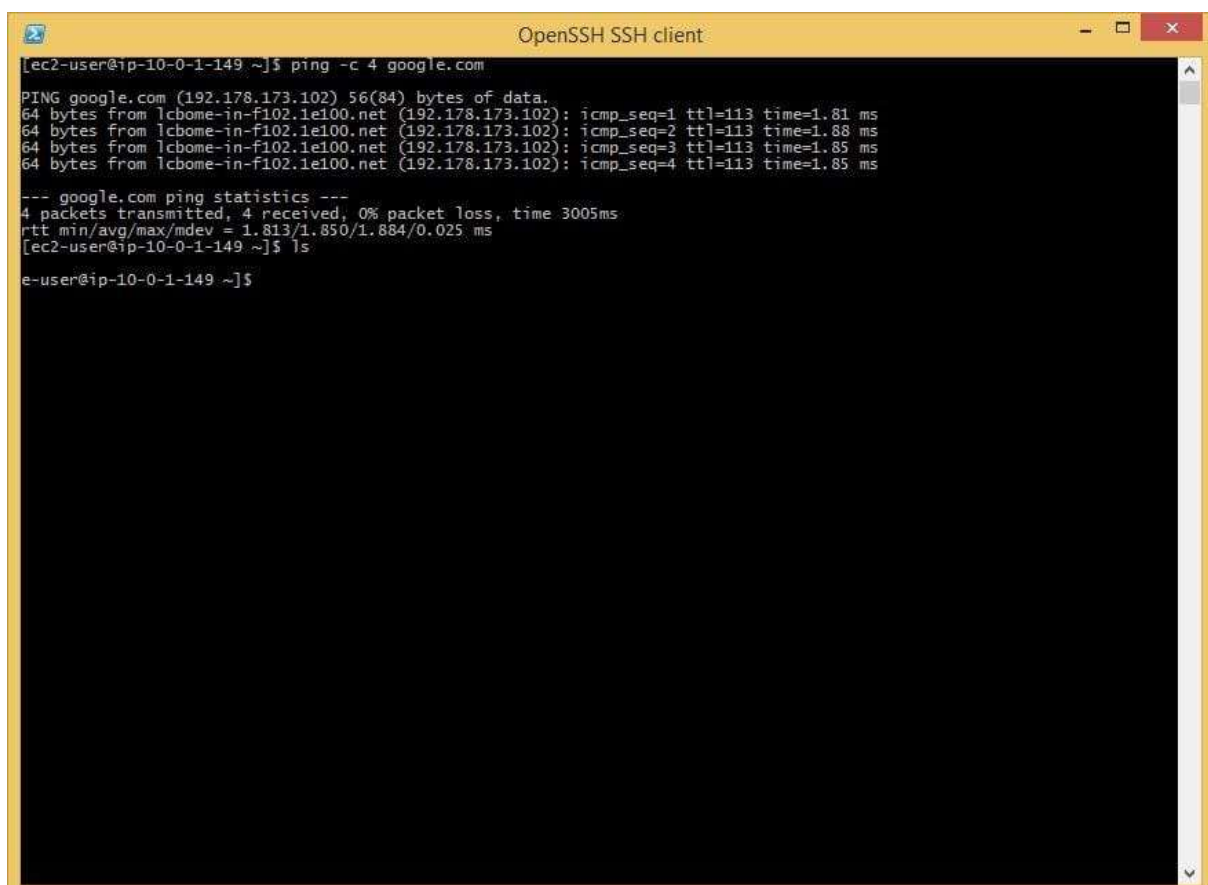
ec2-user@ip-10-0-1-149 ~]$ ping -c 4 google.com
PING google.com (192.178.173.102) 56(84) bytes of data:
64 bytes from lcbome-in-f102.1e100.net (192.178.173.102): icmp_seq=1 ttl=113 time=1.84 ms
64 bytes from lcbome-in-f102.1e100.net (192.178.173.102): icmp_seq=2 ttl=113 time=1.86 ms
64 bytes from lcbome-in-f102.1e100.net (192.178.173.102): icmp_seq=3 ttl=113 time=1.85 ms
64 bytes from lcbome-in-f102.1e100.net (192.178.173.102): icmp_seq=4 ttl=113 time=1.84 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 1.839/1.846/1.862/0.009 ms
```

OUTPUT:

- Successfully created a VPC with Public and Private Subnets.
- Internet Gateway and NAT Gateway were configured successfully.
- Elastic IP was attached to the NAT Gateway.
- Private Route Table was updated to route internet traffic through the NAT Gateway.

FIG: PING GOOGLE.COM



```
[ec2-user@ip-10-0-1-149 ~]$ ping -c 4 google.com
PING google.com (192.178.173.102) 56(84) bytes of data:
64 bytes from lcbome-in-f102.1e100.net (192.178.173.102): icmp_seq=1 ttl=113 time=1.81 ms
64 bytes from lcbome-in-f102.1e100.net (192.178.173.102): icmp_seq=2 ttl=113 time=1.88 ms
64 bytes from lcbome-in-f102.1e100.net (192.178.173.102): icmp_seq=3 ttl=113 time=1.85 ms
64 bytes from lcbome-in-f102.1e100.net (192.178.173.102): icmp_seq=4 ttl=113 time=1.85 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.813/1.850/1.884/0.025 ms
[ec2-user@ip-10-0-1-149 ~]$ ls
e-user@ip-10-0-1-149 ~]$
```


5.CONCLUSION:

SUMMARY-

- Created a VPC with public and private subnets.
- Configured an Internet Gateway and a NAT Gateway.
- Associated route tables correctly.
- Launched EC2 instances in both subnets.
- Verified internet access from the private EC2 instance through the NAT Gateway.

LEARNING OUTCOME-

- Understood the purpose of a NAT Gateway in AWS.
- Learned the difference between Public and Private Subnets.
- Configured Internet Gateway, NAT Gateway, and Route Tables.

THANK YOU
